BIBLIOTECA DE INGENIERÍA CON AGENTES · VOL. 1



El Castor y el Plano

Spec-Driven Development: cómo dirigir agentes de código escribiendo la especificación antes que el código

Una guía didáctica · Edición 2026

ANTES DE EMPEZAR

Cómo leer este libro

Un castor no nace sabiendo construir presas perfectas. Aprende observando el agua, midiendo la corriente y colocando cada tronco con intención. Este libro enseña a programar con esa misma disciplina, ahora que quien coloca los troncos puede ser un agente de inteligencia artificial.

Durante décadas, escribir software fue sinónimo de escribir código. Llegaron los agentes de IA y, por un momento, pareció que bastaba con pedirles las cosas en lenguaje natural: el famoso *vibe coding*. Funciona de maravilla para un prototipo de fin de semana y fracasa estrepitosamente cuando hay que mantener un sistema real en producción. **Spec-Driven Development (SDD)** es la respuesta de la ingeniería a ese problema: volver a poner un plan —la especificación— en el centro, pero un plan vivo, ejecutable, que los agentes puedan leer y obedecer.

El libro está pensado para leerse en orden, aunque cada capítulo se sostiene por sí mismo. Encontrarás tres tipos de recuadros que te acompañarán:

> 🦫 **EL CASTOR DICE**
>
> Metáforas y reglas mnemotécnicas para fijar la idea central de cada capítulo. Cuando veas al castor, estás ante algo que conviene recordar.

> **EN LA PRÁCTICA**
>
> Comandos reales, archivos de ejemplo y fragmentos de especificación que puedes copiar y adaptar a tus proyectos hoy mismo.

> ⚠ **CUIDADO CON ESTO**
>
> Errores frecuentes y malentendidos que hacen que SDD se convierta en burocracia inútil en lugar de una ventaja real.

Cada término técnico aparece **en negrita la primera vez** y está recogido al final, en el Diccionario en lenguaje claro. Si te topas con una palabra que no conoces, búscala ahí: las definiciones están escritas para entenderse sin jerga previa.

MAPA DE LA OBRA

Temario

Tres partes: primero entendemos el porqué, luego dominamos el método paso a paso, y finalmente lo llevamos a un caso real de principio a fin.

PARTE I · POR QUÉ ESPECIFICAR

01 ¿Qué es Spec-Driven Development?

1.1 La especificación como única fuente de verdad · 1.2 Del vibe coding al spec · 1.3 SDD frente a TDD · 1.4 Los tres niveles de rigor · 1.5 La parábola del castor

02 Agentes: el motor que ejecuta el plano

2.1 Qué es un agente de codificación · 2.2 El bucle agéntico · 2.3 Por qué un agente necesita un spec · 2.4 Contexto: la memoria del agente

PARTE II · EL MÉTODO, FASE POR FASE

03 La Constitución del proyecto

3.1 Principios no negociables · 3.2 Cómo redactar una buena constitución · 3.3 Ejemplo comentado

04 Especificar y clarificar

4.1 Anatomía de una especificación · 4.2 Los seis elementos imprescindibles · 4.3 La caza de ambigüedades · 4.4 Ejemplo paso a paso

05 Planificar, descomponer y analizar

5.1 Del qué al cómo · 5.2 Tareas atómicas · 5.3 La revisión previa que evita desastres

06 Implementar y mantener vivo el spec

6.1 Ejecución por fases · 6.2 Cuando los requisitos cambian · 6.3 La revisión humana sigue siendo tuya

PARTE III · MANOS A LA OBRA

07 El ecosistema de herramientas en 2026

7.1 GitHub Spec Kit · 7.2 AWS Kiro · 7.3 Claude Code y las Skills · 7.4 Cómo elegir

08 Caso completo: de la idea al código

Construimos juntos un módulo real siguiendo las siete fases

APÉNDICE

★ Diccionario en lenguaje claro

INTRODUCCIÓN

El día en que el código dejó de ser el centro

A principios de 2025, Andrej Karpathy puso nombre a una forma de programar que ya practicaban millones: el vibe coding. Le pides cosas a una IA, te da código, lo pruebas, falla, vuelves a pedir. Magia pura… hasta que el proyecto crece.

El problema del vibe coding no es la IA. Es la **ausencia de un plan**. Cuando le hablas a un agente solo con prompts sueltos, cada respuesta es una interpretación nueva de lo que crees que quieres. El agente rellena los huecos a su manera, inventa nombres de funciones que no existen —lo que llamamos **alucinación**—, y poco a poco el código se aleja de tu intención original. A esa deriva silenciosa se le llama **drift**, y es la razón por la que un estudio de comienzos de 2026 llegó a contar más de cien mil incidencias introducidas por IA que seguían vivas en repositorios de producción.

La industria reaccionó con una idea vieja vestida de nueva: si el problema es la falta de un plan claro, escribamos el plan primero y hagamos que el agente lo obedezca. Ese plan es la **especificación**, o *spec*. Y la metodología que la coloca en el centro es **Spec-Driven Development**.

La frase que resume todo el movimiento, repetida en los blogs de GitHub y AWS durante 2025 y 2026, es deliciosamente simple: *«la especificación es el prompt»*. Ya no improvisas instrucciones. Escribes un documento estructurado, versionado, que describe qué debe hacer el sistema, y de ahí derivas el plan, las tareas y, por último, el código.

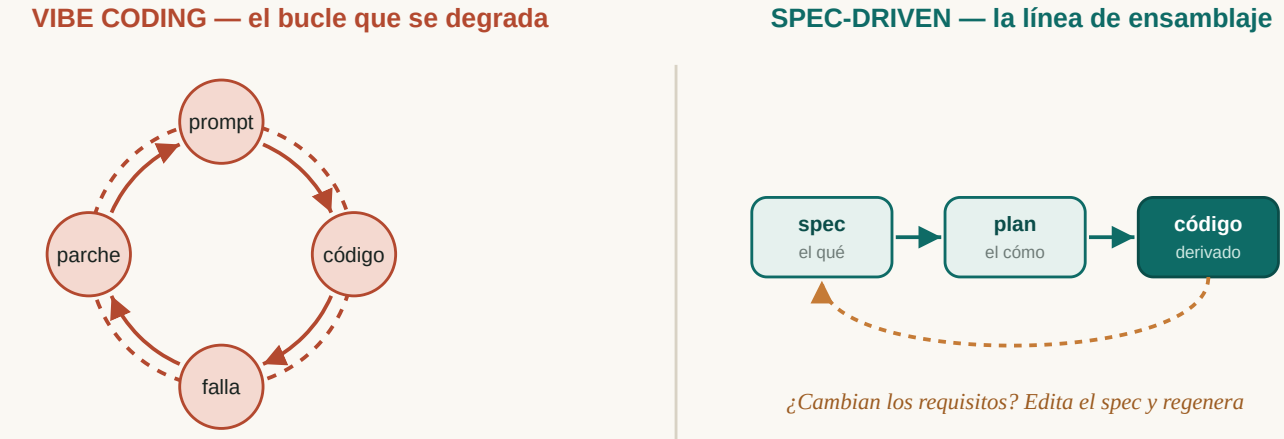


Figura 0.1 · El vibe coding gira sobre sí mismo; SDD avanza en una dirección y trata la especificación como la pieza que regeneras cuando algo cambia.

Para 2026, esto dejó de ser un experimento. Toda herramienta seria de programación con IA —GitHub Spec Kit, AWS Kiro, Claude Code, Cursor, Gemini— incorpora su propia versión de SDD. Spec Kit superó las noventa mil estrellas en GitHub, y plataformas educativas lanzaron cursos dedicados. La pregunta ya no es *si* usar especificaciones, sino *cómo escribirlas bien*. A eso dedicamos el resto del libro.

PARTE I · CAPÍTULO 01

¿Qué es Spec-Driven Development?

Una metodología en la que el código deja de ser el documento principal. El protagonista pasa a ser una especificación ejecutable y versionada, de la que el código se deriva.

1.1 La especificación como única fuente de verdad

En el desarrollo tradicional, el código *es* la verdad. Los documentos de diseño envejecen, los comentarios mienten, pero el código siempre dice lo que el sistema hace realmente. SDD invierte esa relación. Aquí la **fuente de verdad** —el documento que manda cuando hay una discrepancia— es la especificación. El código es una consecuencia de ella.

Esto solo es posible porque ahora tenemos agentes capaces de leer una especificación detallada y producir código a partir de ella. La spec deja de ser «documentación pasiva» que nadie actualiza y se convierte en un **contrato ejecutable**: un texto que no solo describe el sistema, sino que activamente lo gobierna. Si quieres cambiar el comportamiento, cambias la spec y regeneras; no parcheas el código a mano y dejas la documentación desactualizada.

> 🦫 **EL CASTOR DICE**
>
> Un castor nunca empieza colocando troncos al azar. Primero «lee» el río: dónde está la corriente, qué profundidad hay, por dónde escapa el agua. Esa lectura es su especificación. La presa es solo la consecuencia física de haberla entendido bien.

1.2 Del vibe coding al spec: el mismo agente, distinta disciplina

Conviene no demonizar el vibe coding. Para explorar una idea, hacer un prototipo desechable o aprender una librería nueva, prompts sueltos son perfectos: rápidos y sin ceremonia. El problema aparece cuando ese prototipo, sin querer, se convierte en producción. Lo que servía para experimentar empieza a sostener datos reales de usuarios reales, y la falta de estructura se paga cara.

SDD es, literalmente, *lo que viene después del vibe coding* cuando necesitas código que puedas mantener y desplegar con confianza. No es una herramienta distinta: es el mismo agente, trabajando bajo una disciplina.

1.3 SDD frente a TDD: parientes, no rivales

Si conoces el **desarrollo guiado por pruebas (TDD)**, la idea de «escribir algo antes que el código» te resultará familiar. La diferencia está en el nivel. Las pruebas unitarias de TDD verifican funciones

individuales: comprueban que `sumar(2,2)` devuelva `4`. Pero no detectan violaciones de arquitectura, contratos de API que se rompen entre servicios, o decisiones de diseño que contradicen lo acordado. La especificación de SDD opera a nivel de *sistema*: captura clases de errores que las pruebas unitarias, por su naturaleza, no pueden ver.

Dimensión	Vibe coding	TDD	Spec-Driven Development
Artefacto principal	El prompt del momento	Las pruebas	La especificación
Qué se escribe primero	Nada, se improvisa	Una prueba que falla	Qué debe hacer el sistema
Nivel que protege	Ninguno	La función	El sistema completo
Bueno para	Prototipos, exploración	Lógica concreta	Producción y escala
Riesgo principal	Drift y alucinaciones	Cobertura sin visión global	Specs burocráticas e inútiles

No tienes que elegir uno. En la práctica, una buena spec exige pruebas, y SDD y TDD conviven sin problema: el spec dice *qué* y *por qué*, las pruebas confirman que cada pieza cumple su parte.

1.4 Los tres niveles de rigor

No todo proyecto necesita la misma dosis de especificación. El equipo de Martin Fowler popularizó una distinción de tres niveles, según cuánta autoridad le das a la spec frente al código:



Figura 1.1 · Elige tu nivel según el proyecto. Lo más común y sensato hoy es empezar en **spec-anchored**: ni descuidas la spec ni renuncias a tocar el código.

1.5 La parábola del castor

A lo largo del libro, el castor será nuestro modelo de ingeniero. No por simpático, sino porque encarna exactamente lo que SDD pide: un constructor que no improvisa, que entiende el problema antes de actuar, que construye con piezas reutilizables (cada tronco cumple una función) y que, cuando el río cambia de curso, no demuele todo: ajusta el plan y reconstruye lo necesario. Si en algún momento te pierdes entre comandos y artefactos, vuelve a la imagen del castor leyendo el río. Esa es toda la metodología en una estampa.

Agentes: el motor que ejecuta el plano

Una especificación sin nadie que la ejecute es solo un deseo. El que coloca los troncos según el plano es el agente de codificación. Entender cómo «piensa» es entender por qué necesita una buena spec.

2.1 Qué es un agente de codificación

Un **agente de codificación** es un programa que usa un modelo de lenguaje grande (LLM) para realizar tareas de programación de forma semiautónoma: leer archivos, escribir código, ejecutar comandos, revisar los resultados y corregirse. La diferencia con un simple chat es la **autonomía**: un chat te responde y espera; un agente actúa, observa qué pasó y decide el siguiente paso por su cuenta, repitiendo el ciclo hasta cumplir el objetivo. Claude Code, Cursor o los agentes de Copilot son ejemplos.

2.2 El bucle agéntico

Todo agente, por dentro, gira en un mismo ciclo de cuatro movimientos. Reconocerlo te ayuda a entender dónde encaja la especificación y por qué, sin ella, el ciclo se descontrola.

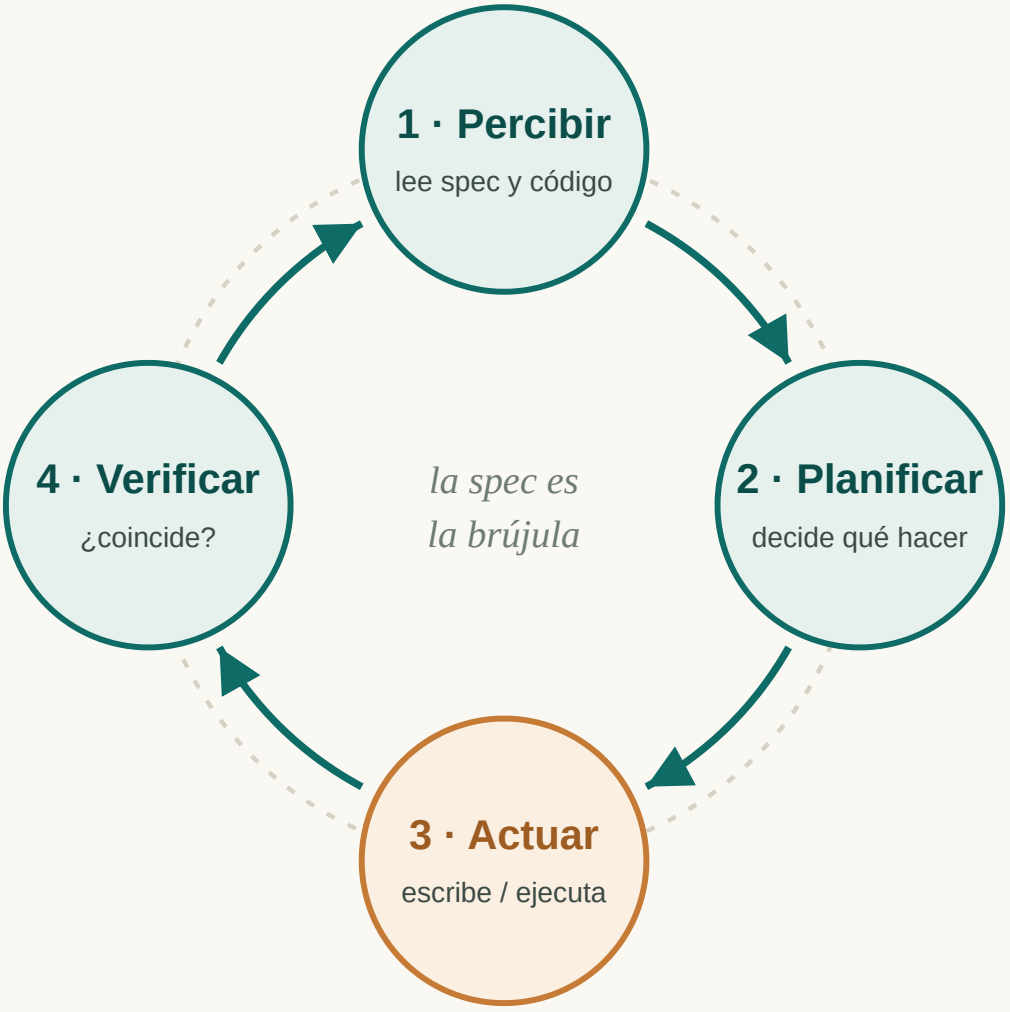


Figura 2.1 · El bucle agéntico. En el paso 4, «¿coincide?» significa *¿coincide con la especificación?*. Sin spec, el agente verifica contra su propia interpretación, que va cambiando en cada vuelta.

2.3 Por qué un agente necesita un spec

Aquí está el corazón del asunto. Tanto un humano como un agente rellenan los huecos de un requisito ambiguo. La diferencia es *cómo* los rellenan. Una persona usa el contexto de la organización, sabe qué se hizo antes, intuye qué espera el jefe. El agente también rellena huecos, pero no como tú querrías: completa con lo más probable estadísticamente, no con lo que tu negocio necesita. Una spec bien escrita es, en el fondo, un mecanismo para **no dejar huecos peligrosos**.

> ⚠ **CUIDADO CON ESTO**
>
> Un mito común dice que con SDD «cualquiera puede programar escribiendo specs». Es falso. La spec no sustituye el criterio técnico: sigues necesitando saber cómo es un buen esquema de base de datos, reconocer cuándo la salida del agente tiene un error sutil y entender tus patrones de arquitectura. La spec es una herramienta de comunicación, no un reemplazo de la experiencia.

2.4 Contexto: la memoria del agente

La especificación es una parte del **contexto** que recibe el agente, pero no la única. El contexto también incluye las convenciones del proyecto, la arquitectura existente y lo que el agente «aprendió» en sesiones anteriores — su memoria del código. Obsesionarse con la spec perfecta mientras ignoras el resto del contexto es como escribir una receta detalladísima y olvidarte de surtir la cocina. La spec importa, pero no es lo único que importa: es la pieza que formaliza el *qué* y el *por qué*, dentro de un contexto más amplio que también debes cuidar.

La Constitución del proyecto

Antes de especificar la primera funcionalidad, defines las reglas que ninguna funcionalidad podrá romper. Eso es la constitución: la ley fundamental de tu proyecto.

El método SDD, tal como lo implementan herramientas como GitHub Spec Kit, sigue una secuencia de fases encadenadas. Cada fase produce un artefacto —un archivo— que alimenta a la siguiente, como estaciones de una línea de ensamblaje: entra una idea vaga por un extremo y sale código probado por el otro. Este es el panorama completo que iremos desglosando en los próximos capítulos:

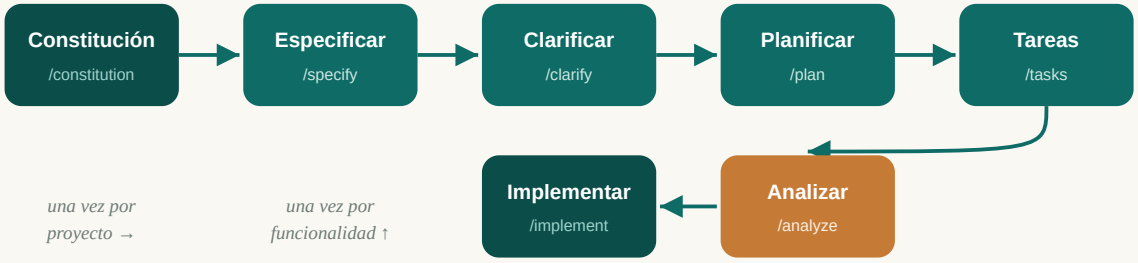


Figura 3.1 · El pipeline SDD. La **constitución** se escribe una vez para todo el proyecto; el resto del ciclo se repite por cada funcionalidad. *Analizar* (en ámbar) es la auditoría previa que evita implementar sobre cimientos torcidos.

3.1 Principios no negociables

La **constitución** es un documento que recoge los principios innegociables de tu proyecto: reglas que toda funcionalidad debe respetar, pase lo que pase. No describe *qué* construyes, sino *cómo* debe construirse siempre. Por ejemplo: «toda entrada de usuario debe validarse», «usamos arquitectura de microservicios», «cada funcionalidad nace como una librería reutilizable antes de integrarse».

Su utilidad real aparece en la fase de análisis: el agente compara el plan y las tareas contra la constitución, y marca como **crítico** cualquier punto que la viole. Es tu forma de codificar el criterio de tu equipo una sola vez para que se aplique en cada funcionalidad, sin repetirlo en cada prompt.

3.2 Cómo redactar una buena constitución

Una buena constitución es corta, concreta y verificable. Evita los buenos deseos vagos («el código debe ser limpio») y prefiere reglas comprobables («ninguna función supera 50 líneas», «toda consulta a base de datos usa parámetros, nunca concatenación de cadenas»). Piensa en cada artículo como algo que un revisor podría marcar con un sí o un no.

3.3 Ejemplo comentado

Así se vería el comando para inicializar la constitución de un portal interno de proveedores, con principios de seguridad por delante:

```
# En tu agente (Claude Code, Copilot, etc.), tras instalar Spec Kit:
/speckit.constitution Este proyecto es un portal de proveedores con
  seguridad como prioridad. Principios no negociables:
  1. Toda entrada de usuario se valida en servidor, nunca solo en cliente.
  2. Acceso a datos siempre con consultas parametrizadas (anti inyeccion SQL).
  3. Cada proveedor solo accede a sus propios documentos (control de autorizacion).
  4. Toda accion sensible queda registrada en una bitacora de auditoria.
  5. El codigo se organiza en modulos reutilizables, no en un monolito.
```

El agente convierte esto en un archivo `constitution.md` versionado. A partir de ahí, cada vez que especifiques una funcionalidad nueva, esos cinco principios actúan como guardarraíles automáticos.

> 🦫 **EL CASTOR DICE**
>
> La constitución es el suelo de roca sobre el que el castor ancla la presa. No es la presa, pero sin él todo lo demás se lo lleva la corriente. Se pone una vez, y aguanta toda la obra.

Especificar y clarificar

El corazón del método. Aquí transformas una idea difusa en un documento estructurado del que un agente puede derivar código sin inventarse lo que no le dijiste.

4.1 Anatomía de una especificación

Una spec no es un documento de requisitos (PRD) con otro nombre. Un PRD se escribe para humanos, que interpretan la ambigüedad y rellenan vacíos con contexto organizacional. Una spec se escribe para que un agente la ejecute, así que debe cerrar esos vacíos de forma explícita. La diferencia es sutil pero decisiva.

4.2 Los seis elementos imprescindibles

Una buena especificación define seis cosas. Si falta alguna, el agente la rellenará por ti, y rara vez como esperabas:



Figura 4.1 · Los seis elementos. Los tres de la izquierda definen **el problema**; los tres de la derecha, **cómo reconocer la solución**. El sexto —criterios de aceptación— es el que más se olvida y el que más drift evita.

4.3 La caza de ambigüedades

Las primeras specs casi siempre tienen huecos. Por eso existe una fase intermedia, **clarificar**, pensada justo para cuando la descripción es vaga o incompleta. La herramienta hace un barrido de ambigüedades en categorías como alcance funcional, modelo de datos, flujo de experiencia, casos límite y terminología. Luego te hace hasta cinco preguntas muy concretas, una a una, y escribe tus respuestas de vuelta en la spec. Así ningún acuerdo se pierde.

4.4 Ejemplo paso a paso

Imagina que quieres una funcionalidad de exportación de facturas. Una primera orden vaga sería:

```
/speckit.specify Que el usuario pueda exportar sus facturas.
```

El comando `/speckit.clarify` detectaría enseguida los huecos y preguntaría, por ejemplo:

```
# Preguntas que el agente plantea, una a una:
P1. ¿En que formatos? (CSV, Excel, PDF)
P2. ¿Que rango de fechas puede elegir el usuario?
P3. ¿Que pasa si no hay facturas en el rango elegido?
P4. ¿La exportacion incluye facturas anuladas?
P5. ¿Hay limite de registros por exportacion?
```

Tras responder, la spec deja de ser un deseo y se convierte en un contrato. Fíjate en cómo cada pregunta corresponde a uno de los seis elementos: las P3 y P4 son casos límite (restricciones), la P5 es una restricción técnica, la P1 es parte del resultado. Ese es el valor de clarificar: convierte lo implícito en explícito antes de que el agente lo decida por su cuenta.

> **EN LA PRÁCTICA**
>
> Para experimentos rápidos puedes ir directo: `specify → plan → tasks → implement`. Pero para cualquier trabajo de producción o con ambigüedad real, intercala `clarify`, `checklist` y `analyze`. La regla es simple: a mayor ambigüedad o riesgo, más fases de control.

PARTE II · CAPÍTULO 05

Planificar, descomponer y analizar

Ya sabes qué construir. Ahora el agente decide cómo, lo parte en tareas pequeñas y —antes de tocar una sola línea— auditas que todo encaje.

5.1 Del qué al cómo

El comando `/plan` toma tu spec y la convierte en un **plan de implementación**: el documento que traduce el *qué* en un *cómo* técnico. Aquí el agente lee la spec, relee la constitución para no contradecirla, y produce decisiones de arquitectura, modelos de datos, contratos de API y, a veces, una guía de arranque rápido. Si algo en el plan choca con la constitución, debería detenerse y avisarte.

Una idea clave: la spec y el plan están desacoplados. Puedes reutilizar la misma spec con un plan distinto —por ejemplo, cambiar de base de datos manteniendo intacto el *qué*—. El qué perdura; el cómo es intercambiable.

5.2 Tareas atómicas

El comando `/tasks` desmenuza el plan en una lista de **tareas atómicas**: unidades de trabajo lo bastante pequeñas como para completarse y verificarse de una en una. Cada historia de usuario suele convertirse en su propia fase, con sus tareas. ¿Por qué tan pequeñas? Porque un agente, igual que una persona, comete menos errores con instrucciones acotadas que con un encargo gigante y difuso.

> 🦫 **EL CASTOR DICE**
>
> El castor no carga el bosque entero al río. Corta un tronco, lo coloca, comprueba que sujeta, y va por el siguiente. Tareas atómicas son sus troncos: pequeñas, verificables, una tras otra.

5.3 La revisión previa que evita desastres

Antes de implementar, el comando `/analyze` revisa todos los artefactos —constitución, spec, plan y tareas— buscando incoherencias, ambigüedades o huecos de cobertura. Es de solo lectura: no cambia nada, solo te informa. Produce un informe ordenado por severidad, de **crítico** a bajo.

```
# Informe tipico de /analyze (resumido):
[CRITICO] La tarea T-07 viola el articulo 3 de la constitucion
          (un proveedor podria ver documentos de otro).
[ALTO]    El requisito "exportar a PDF" no tiene ninguna tarea asociada.
[MEDIO]   Ambiguedad: "rango de fechas" no define zona horaria.
[BAJO]    Terminologia inconsistente: "factura" vs "comprobante".
```

La regla de oro: **los problemas críticos se resuelven antes de implementar**. Un crítico casi siempre significa que el agente, de seguir adelante, escribiría código que falla en producción o que viola una regla de seguridad. Diez minutos de análisis aquí ahorran horas de depuración después.

Implementar y mantener vivo el spec

Llega el momento de escribir código. Y, sobre todo, de entender qué hace que SDD valga la pena a largo plazo: la spec no se archiva, se mantiene viva.

6.1 Ejecución por fases

El comando `/implement` recorre la lista de tareas y escribe el código, respetando dependencias y marcas de paralelismo. En proyectos grandes, la recomendación es **implementar por fases** para no saturar el contexto del agente: empieza por la funcionalidad central (la primera historia de usuario), valida que funciona, y solo entonces añades el resto de forma incremental. Cada fase produce algo demostrable y probable.

> **⚠ CUIDADO CON ESTO**
>
> No trates la spec como una garantía. Que el agente haya seguido un documento impecable no te exime de revisar su salida con el mismo rigor que tendrías sin spec. La especificación reduce los errores; no los elimina. El último filtro sigues siendo tú.

6.2 Cuando los requisitos cambian

Aquí está la diferencia que justifica todo el método. En el desarrollo tradicional, cuando cambia un requisito, modificas el código y, con suerte, alguien actualiza la documentación (normalmente no). En SDD, **editas la spec y regeneras la parte afectada**. La spec nunca queda obsoleta porque es el punto de partida, no un subproducto. A esto nos referimos cuando decimos que la spec está «viva»: es memoria persistente y contexto compartido entre tú y el agente, sesión tras sesión.



Nunca parcheas el código directamente: la fuente de verdad es la spec.

Figura 6.1 · El ciclo de mantenimiento en SDD. El código es siempre *aguas abajo* de la spec.

6.3 La revisión humana sigue siendo tuya

SDD no automatiza tu juicio; lo enfoca. Al sacarte de la tarea de teclear cada línea, te libera para concentrarte en lo que de verdad aporta valor: decidir *qué* debe hacer el sistema y *por qué*, y verificar que el resultado lo cumple. El esfuerzo humano se desplaza de escribir código a definir comportamientos con precisión. Esa, en una frase, es la promesa de Spec-Driven Development.

El ecosistema de herramientas en 2026

El método es independiente de la herramienta, pero conviene conocer a los protagonistas. Para mediados de 2026 había más de treinta marcos en este ecosistema; estos son los que marcan el rumbo.

7.1 GitHub Spec Kit

Spec Kit es la opción de código abierto más popular, con más de noventa mil estrellas en GitHub. Curiosamente, sus comandos no son ejecutables tradicionales: son **plantillas en formato Markdown** que tu agente lee e interpreta. Cuando escribes `/speckit.specify`, el agente carga el archivo de instrucciones de ese comando y lo sigue. Funciona con más de veinte plataformas de agentes —Claude Code, Copilot, Gemini CLI, Amazon Q y muchas más— porque, al final, solo son prompts estructurados. Se instala así:

```
# Instalar la CLI y crear un proyecto integrado con tu agente
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
specify init mi-proyecto --integration claude
```

Su gran ventaja es la personalización: con *extensiones* añades fases nuevas (integración con Jira, revisión de seguridad, trazabilidad regulatoria) y con *presets* cambias cómo trabaja sin añadir capacidades —por ejemplo, forzar un formato de spec orientado a cumplimiento, o traducir todo el flujo a otro idioma.

7.2 AWS Kiro

Kiro es un entorno de desarrollo (IDE) construido enteramente alrededor del flujo spec-driven. En lugar de añadir SDD a un editor existente, lo hornea en el producto: codifica el ciclo completo con pasos de constitución, especificación, planificación, tareas e implementación como ciudadanos de primera clase. AWS ha documentado casos de clientes donde funcionalidades que tomaban cuarenta horas se entregaron en menos de ocho horas de trabajo humano cuando se escribieron como spec primero.

7.3 Claude Code y las Skills

Claude Code es el agente de codificación de Anthropic, que opera desde la terminal o el editor y se integra de forma nativa con Spec Kit. Su pieza distintiva son las **Skills**: carpetas con instrucciones y recursos que enseñan al agente a hacer bien una tarea concreta y repetible —desde generar un tipo de documento hasta

seguir las convenciones internas de un equipo—. Encajan de forma natural con SDD: una Skill puede encapsular «cómo escribimos nosotros las specs» y aplicarse en cada proyecto.

7.4 Cómo elegir

Si tú…	Empieza con	Por qué
Ya usas un agente y quieres añadir disciplina	Spec Kit	Se monta encima de lo que ya tienes, sin cambiar de herramienta
Quieres una experiencia integral guiada	AWS Kiro	El flujo SDD viene incorporado de fábrica
Trabajas desde la terminal y valoras la flexibilidad	Claude Code + Spec Kit	Máximo control y Skills reutilizables
Solo quieres entender el método	Cualquiera	Las fases son las mismas en todas; cambia el envoltorio

> **EN LA PRÁCTICA**
>
> No te cases con una herramienta antes de dominar el método. Las siete fases —constitución, especificar, clarificar, planificar, tareas, analizar, implementar— son universales. Si las entiendes, migrar entre Spec Kit, Kiro o cualquier otra es cuestión de aprender comandos nuevos, no conceptos nuevos.

Caso completo: de la idea al código

Retomamos la exportación de facturas del capítulo 4 y la llevamos de punta a punta. Una funcionalidad pequeña, las siete fases reales, para que veas el método en movimiento.

El encargo: «Necesitamos que cada usuario pueda exportar sus facturas a Excel para presentarlas ante el organismo tributario». Así de vago empieza casi todo. Veamos cómo SDD lo convierte en código fiable.

Fase 1 · Constitución (ya existe)

Nuestro proyecto ya tiene la constitución del capítulo 3. Sus artículos sobre validación, autorización por usuario y bitácora de auditoría se aplicarán automáticamente. No la reescribimos: la heredamos.

Fase 2 · Especificar

```
/speckit.specify Exportacion de facturas a Excel. El usuario autenticado
  descarga sus propias facturas en un archivo .xlsx para tramites tributarios.
```

Fase 3 · Clarificar

El agente detecta huecos y pregunta. Tras responder, la spec recoge las decisiones:

```
# Fragmento de spec.md tras clarificar
## Clarificaciones
- Formato: solo .xlsx (una hoja, encabezados en fila 1).
- Rango: el usuario elige fecha inicio y fin; maximo 12 meses.
- Sin resultados: se descarga igual, con encabezados y aviso "Sin facturas".
- Anuladas: se incluyen, con columna Estado = ANULADA.
- Limite: 5000 filas por exportacion; mas alla, se sugiere acotar el rango.
- Zona horaria: las fechas se interpretan en hora de Lima (UTC-5).
```

Fase 4 · Planificar

```
# Fragmento de plan.md
Endpoint:   GET /api/facturas/exportar?desde=&hasta=
Auth:       token de sesion; filtra por id_usuario del token (art. 3)
Datos:      consulta parametrizada a tabla facturas (art. 2)
Libreria:   generacion de .xlsx en streaming (no cargar todo en memoria)
Auditoria:  registrar exportacion en bitacora (art. 4)
```

Fase 5 · Tareas

```
# Fragmento de tasks.md
T-01 Validar parametros de fecha (rango, maximo 12 meses).
T-02 Consulta parametrizada de facturas del usuario en el rango.
T-03 Generar .xlsx en streaming con las columnas definidas.
T-04 Caso sin resultados: archivo con aviso.
T-05 Registrar la exportacion en la bitacora de auditoria.
T-06 Pruebas: rango invalido, usuario ajeno, limite de filas.
```

Fase 6 · Analizar

```
[MEDIO] T-03 no especifica que hacer si una factura tiene moneda distinta.
[BAJO]  Falta definir el nombre del archivo descargado.
```

Ningún crítico: la autorización por usuario (artículo 3) ya está cubierta en T-02. Resolvemos los dos puntos menores ajustando la spec —añadimos columna Moneda y el patrón de nombre `facturas_AAAA-MM-DD.xlsx` — y seguimos.

Fase 7 · Implementar

Con todo verificado, `/speckit.implement` ejecuta las tareas en orden. El agente escribe el endpoint, la consulta parametrizada, el generador de Excel, el manejo del caso vacío, el registro de auditoría y las pruebas. Tú revisas el resultado —porque la revisión sigue siendo tuya— y confirmas que cumple cada criterio de aceptación.

> 🦫 **EL CASTOR TERMINA LA PRESA**
>
> Fíjate en lo que *no* pasó: el agente nunca tuvo que adivinar la zona horaria, ni si incluir anuladas, ni qué hacer sin resultados. Todo estaba en la spec. El código no fue una sorpresa: fue la consecuencia previsible de un buen plano. Eso es construir como un castor.

Y cuando dentro de tres meses el organismo tributario exija una columna nueva, no buscarás en el código dónde tocar. Editarás la spec, añadirás la columna, regenerarás. La presa se adapta al río sin demolerse.

APÉNDICE

Diccionario en lenguaje claro

Cada término técnico del libro, explicado como se lo contarías a alguien que empieza. Sin jerga que necesite más jerga.

Spec-Driven Development (SDD) `desarrollo guiado por especificaciones`

Forma de programar en la que primero escribes un documento detallado de qué debe hacer el sistema (la especificación) y luego el código se deriva de él. El documento manda; el código obedece.

Especificación (o spec)

Documento estructurado que describe qué debe hacer una funcionalidad: sus resultados, sus límites, sus reglas y cómo saber que está terminada. En SDD es la pieza central, no un papel que nadie lee.

Vibe coding `programar "a la intuición"`

Pedirle cosas a una IA con instrucciones sueltas en lenguaje natural, sin plan previo. Genial para prototipos rápidos; arriesgado para sistemas reales que hay que mantener.

Agente de codificación

Programa que usa una IA para programar de forma semiautónoma: lee archivos, escribe código, ejecuta comandos y se corrige solo, repitiendo el ciclo hasta lograr el objetivo. Ejemplos: Claude Code, Cursor, los agentes de Copilot.

LLM `large language model · modelo de lenguaje grande`

El "cerebro" de IA que entiende y genera texto (y código). Es lo que hay dentro de un agente. Predice qué viene a continuación a partir de todo lo que ha leído.

Alucinación

Cuando la IA inventa algo que suena plausible pero es falso: una función que no existe, un dato incorrecto. No miente a propósito; simplemente rellena un hueco con lo que le pareció probable.

Drift `deriva`

El alejamiento gradual entre lo que querías y lo que el código realmente hace, que se acumula cuando das instrucciones sueltas sin un plan firme. Cada interpretación nueva lo aleja un poco más de tu intención.

Fuente de verdad `source of truth`

El documento o artefacto que tiene la última palabra cuando hay una contradicción. En el desarrollo tradicional es el código; en SDD es la especificación.

Contrato ejecutable

Una especificación que no solo describe el sistema, sino que un agente puede leer y convertir en código. "Ejecutable" porque la máquina actúa a partir de ella, no solo la archiva.

TDD `test-driven development · desarrollo guiado por pruebas`

Metodología donde escribes la prueba antes que el código, función por función. SDD trabaja a un nivel más alto (el sistema completo); ambos se complementan.

Bucle agéntico

El ciclo de cuatro pasos que repite un agente: percibir (leer), planificar (decidir), actuar (escribir/ejecutar) y verificar (comprobar). La especificación es la brújula que orienta ese ciclo.

Contexto

Toda la información que el agente tiene presente al trabajar: la spec, las convenciones del proyecto, la arquitectura y lo aprendido en sesiones previas. La spec es una parte del contexto, no todo.

Constitución

Documento con los principios no negociables del proyecto: reglas que toda funcionalidad debe cumplir siempre (seguridad, arquitectura, estilo). Se escribe una vez para todo el proyecto.

Plan de implementación

El documento que traduce el "qué" de la spec en un "cómo" técnico: decisiones de arquitectura, modelos de datos, contratos de interfaz. Se genera tras especificar.

Tarea atómica

Una unidad de trabajo lo bastante pequeña como para completarse y verificarse de una sola vez. Dividir en tareas atómicas reduce los errores, igual en personas que en agentes.

Severidad `crítico · alto · medio · bajo`

Escala con la que la fase de análisis ordena los problemas detectados. Los críticos —violaciones de la constitución o requisitos sin cubrir— deben resolverse antes de escribir código.

Criterios de aceptación

La lista de condiciones que deben cumplirse para considerar terminada una funcionalidad. Es el sexto elemento de una spec y el que más se olvida; sin él, nadie sabe cuándo "está listo".

Spec-first / spec-anchored / spec-as-source

Los tres niveles de rigor según cuánta autoridad tiene la spec frente al código: dirige solo el arranque (first), conviven sincronizados (anchored), o es el único artefacto que editas y el código es desechable (as-source).

GitHub Spec Kit

Conjunto de herramientas de código abierto para SDD. Sus comandos son plantillas de texto que tu agente interpreta. Compatible con muchas plataformas de agentes.

AWS Kiro

Entorno de desarrollo (IDE) de Amazon construido enteramente alrededor del flujo spec-driven, con las fases incorporadas de fábrica.

Claude Code · Skills

Claude Code es el agente de codificación de Anthropic. Las Skills son carpetas con instrucciones que le enseñan a hacer bien una tarea repetible y se reutilizan entre proyectos.



Lee el río. Dibuja el plano. Luego construye.

Fin de *El Castor y el Plano*. Ahora te toca a ti escribir la primera especificación de tu próximo proyecto.